6

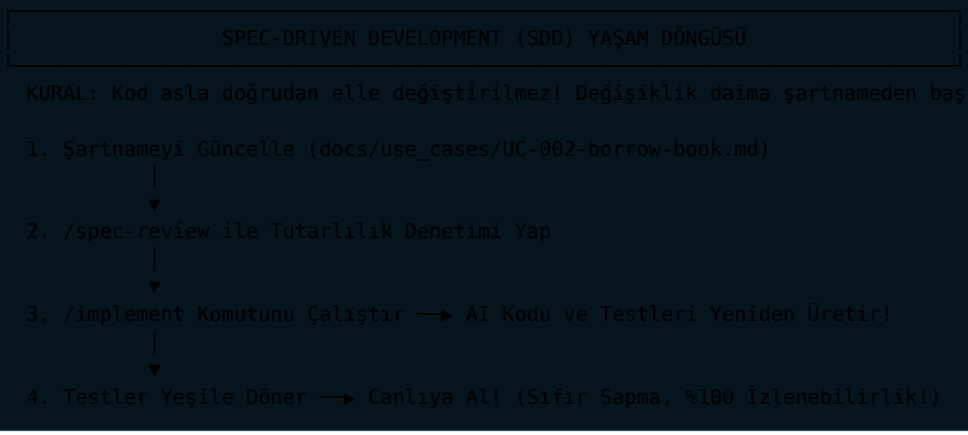
BÖLÜM: BÜYÜK TERSİNE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

# Spec-Driven Development (SDD): Şartnameyi Birincil Varlık Olarak Konumlandırma

Spec-Driven Development: Treating the Specification as the Primary Asset

## ⚡ Akış, Varlık & Süreç Şeması

## ⚡ Process & Artifact Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Bir gökdelen inşa ederken elektrik tesisatını beğenmediğinde gidip duvarı kırıp kafana göre kablo çekmezsin. Önce mimari projeyi (şartnameyi) güncellersin, mühendisler onaylar, sonra ustalar gelip plana göre tesisatı döşer. Eğer projeyi güncellemeden duvara kablo çekersen, yarın başka bir usta gelip o kabloyu keser ve bina yanar! SDD tam olarak budur: Önce planı düzelt, işçiler (AI) plana göre binayı saniyeler içinde yenilesin!

### 🇬🇧 ENGLISH (ELI5)

When building a skyscraper, if you decide to add an electrical outlet, you don't secretly chisel into a load-bearing column and solder wires. You update the architectural blueprints first. Once approved, the construction crew executes the change. Modifying code directly while leaving the specification stale is architectural vandalism; the next automated regeneration will wipe out your undocumented hotfix.

## 🔍 MAANG & AIUP Seviyesi Teknik Derinlik

### 🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Spec-Driven Development (SDD) ve Kodun Geçiciliği Paradigması: Simon Martinelli'nin 'Spec-Driven Development' kitabında altını çizdiği en temel disiplin: Kod artık kalıcı bir varlık değildir; şartnamenin bir derleme çıktısıdır. Geleneksel yaklaşımlarda kod değiştikçe döküman eskir ve çöp olurdu. SDD bu döngüyü kırar: 1) Tek Doğruluk Kaynağı (Single Source of Truth): Şartname ve kod çeliştiğinde her zaman kod hatalı kabul edilir. 2) Yeniden Üretilebilirlik (Regenerability): Yeni bir framework sürümü çıktığında (örneğin Spring Boot 3'ten 4'e veya Java 21'den 25'e), şartname hiç değişmeden tek bir `/implement` komutuyla tüm codebase modern kütüphanelerle baştan üretilebilir. 3) İzlenebilirlik (Traceability): Canlı sistemdeki her bir satır kod, şartnamedeki bir iş kuralına (BR-XXX) ve use case adımına doğrudan bağlanır.

## 🔍 MAANG & AIUP Technical Deep-Dive

### 🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Spec-Driven Development (SDD) architectural discipline: SDD posits that software source code is no longer a durable asset; it is an ephemeral compilation artifact derived from formal specifications. The core tenets: 1) Specification Primacy: Whenever source code deviates from the use case specification, the code is definitively defective by definition. 2) Effortless Regeneration: Modernizing frameworks or porting stacks (e.g. Java to TypeScript, JPA to jOOQ) becomes mechanical: rerun generation commands against unchanged, human-verified specifications. 3) Strict Traceability: Every generated controller, entity validation, and integration test links back to a specific business rule identifier (BR-XXX).

### ⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Spec-Driven Kod Üretiminin İdeal Çıktısı (Tam İzlenebilir Sınıf)
package com.maang.aiup.sdd;

/**
 * ⚠ DİKKAT: Bu sınıf Spec-Driven Development kapsamında AI tarafından üretilmiştir.
 * Bu dosyayı doğrudan elle DEĞİŞTİRMEYİN!
 * Değişiklikler için şartnameyi güncelleyin: docs/use_cases/UC-002-borrow-book.md
 *
 * Traces to:
 * - Use Case: UC-002 (Borrow Book)
 * - Requirements: FR-012, NFR-002
 * - Rules: BR-004 (Availability), BR-005 (Loan Limit), BR-006 (Overdue Block)
 */
public final class BorrowBookService {
    public record LoanConfirmation(String loanId, String memberId, String bookCopyId,
java.time.LocalDate dueDate) {}

    public LoanConfirmation processLoan(String memberId, String bookCopyId, int activeLoans, boolean
hasOverdue, int availableCopies) {
        // Step 4: System validates that a copy is available (BR-004)
        if (availableCopies <= 0) {
            throw new IllegalStateException("BR-004: Kitap kopyası mevcut değil.");
        }
        // Step 5: System validates member has not reached limit (BR-005)
        if (activeLoans >= 5) {
            throw new IllegalStateException("BR-005: Üye 5 olan ödünç limitine ulaştı.");
        }
        // Step 6: System validates member has no overdue loans (BR-006)
        if (hasOverdue) {
            throw new IllegalStateException("BR-006: Gecikmiş borçlar nedeniyle ödünç verilemez.");
        }

        // Step 7: System records loan with due date 28 days from today
        var dueDate = java.time.LocalDate.now().plusDays(28);
        return new LoanConfirmation("LN-" + System.currentTimeMillis(), memberId, bookCopyId,
dueDate);
    }

    void main() {
        var service = new BorrowBookService();
        var loan = service.processLoan("MEM-1", "CPY-99", 2, false, 3);
        System.out.println("✅ SDD ile üretilen ve şartnameye %100 uyan kayıt: " + loan);
    }
}
```

### ⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

#### 🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Acil bir canlı sistem hatasında (hotfix) kodu doğrudan IDE'den düzeltip 'şartnameyi sonra güncellerim' demek metodolojinin ölüm fermanıdır. O şartname asla güncellenmez ve sonraki kod üretiminde hotfix'iniz tamamen silinir!

#### 🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Applying a hotfix directly to production code while promising to 'update the spec later' destroys the entire methodology. The specification rots immediately, and the next automated regeneration erases your hotfix without warning.

### 🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Spec-Driven Development'ta kod değil, şartname temel varlıktır; kod onun derlenmiş çıktısıdır.

• Kodu doğrudan elle değiştirmek yasaktır; tüm değişiklikler şartnameden başlayarak koda akar.

• Şartnameleri yaşayan birer sözleşme olarak korumak, projeyi gelecekte sıfır maliyetle modernize etmeyi sağlar.

### 🎯 Key Architectural Takeaways

• In Spec-Driven Development, the specification is the primary asset; code is its compiled output.

• Never modify code directly; all changes must cascade downstream from the specification.

• Living specifications allow entire software systems to be safely regenerated onto new tech stacks.